

Design and Implementation of a Blockchain-Based Smart Contract for Secure Authentication and Trusted Data Recording in VANETs

Nour Mohamed, Mohamed Hesham, and Mohamed Tamer

Arab Academy for Science, Technology and Maritime Transport, Computing and Information Technology College, Egypt,

nourmohamed0027@gmail.com, mohamedhesham172839@gmail.com, mt0991787@gmail.com

Abstract— *Vehicular Ad-hoc Networks (VANETs) demand robust security mechanisms to ensure communication integrity and prevent malicious activities such as identity spoofing and data tampering. This paper presents a decentralized framework for Secure Authentication and Trusted Data Recording in VANETs using Ethereum Blockchain technology. The proposed system utilizes a Smart Contract developed in Solidity to manage network entities, enforcing strict access control for a Traffic Authority to register vehicles and for authenticated nodes to report traffic incidents. The solution was implemented and tested using Remix IDE, Ganache for local blockchain simulation, and MetaMask for transaction management. Experimental evaluations demonstrate the system's effectiveness in mitigating impersonation and modification attacks, ensuring that accident data—including location, timestamp, and forensic hashes—remains immutable and verifiable. Performance analysis focuses on gas consumption for deployment and transaction execution, proving the feasibility of the proposed scheme. Compared to traditional centralized architectures, this automation significantly enhances trust, ensures non-repudiation, and provides a tamper-proof ledger for forensic analysis in intelligent transportation systems.*

Keywords— VANET, Blockchain, Smart Contracts, Ethereum, Solidity, Secure Authentication, Trusted Data Recording, Data Integrity, Remix IDE.

I. INTRODUCTION

The rapid advancement of Intelligent Transportation Systems (ITS) has positioned Vehicular Ad-hoc Networks (VANETs) as a cornerstone for modern road safety and traffic efficiency. By enabling real-time Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) communication, VANETs facilitate critical applications such as collision avoidance, traffic congestion management, and emergency response coordination. However, the open and decentralized nature of these networks makes them highly susceptible to malicious cyber threats. Unlike traditional wired networks, VANETs operate in a highly dynamic environment with fleeting connections, requiring robust security mechanisms for identity verification, message integrity, and behavioural accountability.

Critical security challenges in VANETs—such as Sybil attacks, impersonation, and message falsification—pose severe risks to human life and infrastructure. For instance, a malicious actor could broadcast fake accident alerts to cause traffic jams or hide real incidents to obstruct emergency services. In traditional architectures, trust is often managed through centralized Public Key Infrastructures (PKI) and Certificate Authorities (CAs). While

effective to an extent, these centralized approaches suffer from significant limitations: they introduce a single point of failure (SPoF), struggle with scalability in highly dynamic network topologies where vehicles join and leave rapidly, and often incur high computational overhead that hinders the low-latency requirements of vehicular communications. Furthermore, centralized servers are prime targets for Distributed Denial of Service (DDoS) attacks, which can paralyze the entire traffic management system.

Another critical yet often overlooked aspect of VANET security is the integrity of post-incident data. In current systems, accident logs and sensor data are typically stored in local On-Board Units (OBUs) or mutable cloud databases. These storage mediums are vulnerable to manipulation; a dishonest driver involved in a collision might attempt to alter speed logs, timestamps, or location data to evade liability or insurance premiums. Consequently, there is a pressing need for a **"Digital Black Box"**—a system that guarantees the provenance and immutability of forensic evidence from the moment it is generated, ensuring that data cannot be retroactively modified by any party, including system administrators.

To address these limitations and enhance trust in emerging networks, Blockchain technology offers a promising decentralized alternative. Its immutable ledger and cryptographic security features provide a transparent and tamper-proof environment for data exchange. By leveraging **Smart Contracts**, business logic can be automated to enforce strict access control and data validation without the need for a central intermediary. Unlike passive databases, Smart Contracts act as autonomous agents that execute predefined rules deterministically, ensuring that security policies are applied uniformly across the network.

This paper presents a decentralized framework for **Secure Authentication and Trusted Data Recording in VANETs**, utilizing the Ethereum Blockchain ecosystem. The proposed solution employs a Solidity-based Smart Contract designed to manage network entities and govern data reporting. The system ensures that only authorized Traffic Authorities can register vehicles, and only authenticated vehicles can report traffic events. Once an event—such as an accident or road hazard—is recorded, its details (including location, timestamp, and forensic data hash) become immutable, providing a reliable audit trail for forensic

investigations and insurance claims. The implementation utilizes **Remix IDE** for development, **Ganache** for local blockchain simulation, and **MetaMask** for secure transaction management.

The main contributions of this work are as follows:

1. The design and implementation of a secure Smart Contract architecture that enforces role-based access control (RBAC) to prevent impersonation attacks.
2. The development of a trusted data recording mechanism that ensures the integrity and non-repudiation of traffic incident reports on the blockchain ledger.
3. Empirical evaluation of the proposed framework using the Remix and Ganache testbed, analyzing gas consumption and system performance to demonstrate its feasibility for real-world deployment.

II. RELATED WORK

Security in Vehicular Ad-hoc Networks (VANETs) has traditionally relied on Public Key Infrastructure (PKI) and centralized Certificate Authorities (CAs) to manage vehicle identities and secure communication. In these architectures, vehicles authenticate themselves using digital certificates issued by a central trusted third party. While effective in static environments, centralized approaches face significant challenges in the highly dynamic VANET ecosystem. They introduce a single point of failure (SPoF), suffer from scalability bottlenecks when handling millions of vehicular nodes, and incur high communication latency due to the overhead of certificate revocation and management. Furthermore, dependency on a central server makes the network vulnerable to Denial of Service (DoS) attacks, potentially paralyzing critical traffic safety applications.

To mitigate the limitations of centralized architectures, recent research has shifted towards decentralized solutions leveraging Blockchain technology. Blockchain offers a distributed ledger that ensures data transparency and immutability without relying on a central authority. Early implementations utilized Bitcoin-based protocols or Proof-of-Work (PoW) consensus mechanisms to secure V2V communication. However, these approaches are often unsuitable for resource-constrained vehicular environments due to their high computational requirements (mining), excessive energy consumption, and significant transaction latency, which contradicts the real-time requirements of safety-critical messages in VANETs.

The advent of Ethereum and Smart Contracts has introduced programmable logic to blockchain-based VANETs, enabling automated authentication and trust management. Several studies have proposed smart contract-based schemes for reputation management and message verification. Despite their potential, many existing solutions focus primarily on the trustworthiness of the messages rather than the accountability of the nodes themselves. Additionally, few works have addressed the specific

requirement of trusted data recording for forensic purposes—such as securely storing accident details (location, timestamp, and forensic evidence) to prevent tampering by malicious drivers or third parties. Moreover, many proposed frameworks lack a comprehensive evaluation of execution costs (Gas consumption), which is a critical metric for feasibility in Ethereum-based networks.

The framework proposed in this study addresses these gaps by introducing a lightweight, decentralized system for both Secure Authentication and Trusted Data Recording. Unlike generic blockchain solutions, this project leverages a specifically designed Solidity Smart Contract to enforce strict role-based access control (RBAC), ensuring that only authenticated vehicles can participate in the network. Furthermore, it integrates a tamper-proof event logging mechanism that stores critical accident data hashes, ensuring non-repudiation and integrity for post-incident analysis. By utilizing a local Ethereum testbed (Ganache), this work provides a detailed performance evaluation of gas consumption for deployment and transaction execution, demonstrating a practical and efficient alternative to heavy-weight traditional security protocols.

III. SYSTEM MODEL

The operational workflow of the proposed VANET framework is designed as a hybrid architecture that bridges the gap between physical vehicular entities (Off-Chain) and the decentralized Ethereum network (On-Chain). **Fig. 1** illustrates the sequential interaction flow, categorized into two critical phases: System Initialization and Trusted Data Recording. This architecture leverages the distributed nature of the blockchain to eliminate single points of failure, ensuring that traffic data remains transparent, available, and tamper-proof against malicious alterations.

A. Interaction Components and Roles The system architecture comprises four primary entities, each with distinct responsibilities and cryptographic privileges:

1. **Traffic Authority (Admin & Contract Owner):** The Traffic Authority (TA) acts as the central trusted entity responsible for the governance of the VANET system. Upon deploying the smart contract, the TA's wallet address is hardcoded as the contract owner. The TA holds exclusive administrative privileges to invoke state-changing functions, specifically for whitelisting and registering legitimate vehicles. This role effectively functions as the gatekeeper, ensuring that only certified nodes can participate in the network.
2. **Vehicle Driver (User Node):** Vehicles are represented as network nodes equipped with On-Board Units (OBUs) capable of communicating with the blockchain. Each vehicle possesses an Ethereum Externally Owned Account (EOA), managed via a digital wallet. These nodes are authorized to interact with the ledger to report traffic incidents (e.g., accidents or hazards) but are

restricted from accessing administrative functions, enforcing a strict separation of duties.

3. **MetaMask (Cryptographic Middleware):** MetaMask serves as the secure bridge between the user's interface and the Ethereum network. It functions as a Web3 provider that injects the Ethereum API into the interaction environment. Its primary roles include managing the user's private keys locally (Client-Side), cryptographically signing transactions using the Elliptic Curve Digital Signature Algorithm (ECDSA) to ensure non-repudiation, and calculating precise Gas fees to guarantee transaction execution.
4. **Smart Contract (Autonomous Logic):** The core of the system is the VanetSystem smart contract, a Turing-complete program deployed on the Ethereum Virtual Machine (EVM). It encapsulates the business logic, validates permissions using custom modifiers, and manages the state storage of Vehicle structs and TrafficEvent logs. Being immutable after deployment, it guarantees that the rules of the network cannot be manipulated.

B. Workflow and Execution Logic

1) Phase 1: System Setup and Secure Authentication The lifecycle of the system commences with the **Deployment Phase**. The Traffic Authority compiles the Solidity source code into Bytecode and Application Binary Interface (ABI). A contract creation transaction is then broadcast to the network. Upon mining, the contract is instantiated on the EVM, and the TA's address is permanently stored as the state variable `trafficAuthority`. Following deployment, the **Vehicle Registration** process begins. To authenticate a new node, the TA invokes the `registerVehicle()` function, passing the driver's public address and license plate data.

- **Internal Verification:** The Smart Contract executes the `onlyAuthority` modifier, which compares `msg.sender` with the stored `trafficAuthority` variable.
- **State Update:** Upon successful verification, the vehicle's data is stored in a mapping(`address => Vehicle`) data structure, and its boolean flag `isRegistered` is set to true.
- **Event Emission:** Finally, a `VehicleRegistered` event is emitted, generating an on-chain log that allows external applications to index and verify the authenticated vehicle.

2) Phase 2: Trusted Data Recording and Forensic Logging

In this operational phase, an authenticated driver witnesses a traffic incident and attempts to upload the critical forensic data (Event Type, Location Coordinates, and Data Hash) using the `reportEvent()` function. This process ensures data integrity through the following rigorous steps:

- **Transaction Signing & Gas Estimation:** The request is intercepted by MetaMask. The driver is prompted to sign the transaction payload with their private key. This signature proves the origin of the message (Authentication). Simultaneously, a gas limit is estimated to prevent infinite loops, protecting the network from Denial of Service (DoS) attacks.

- **On-Chain Access Control:** Once the transaction is mined, the Smart Contract executes the `onlyRegistered` modifier. This logic queries the internal vehicles mapping. If the sender's address returns `isRegistered == false`, the execution is immediately halted.
- **Execution Logic (Conditional Block):**
 - **If Authorized:** The contract accepts the input parameters and constructs a new `TrafficEvent` memory struct. It automatically generates a timestamp using the `block.timestamp` global variable to temporally seal the record. This struct is then pushed to the immutable events array storage. A success message is returned, and an `EventReported` log is broadcasted to the network.
 - **If Unauthorized:** The contract triggers a `REVERT` opcode. This action halts execution, reverts all state changes to their previous state, and refunds the remaining gas to the sender. This mechanism effectively mitigates impersonation attacks by ensuring that no invalid data is ever written to the permanent ledger.

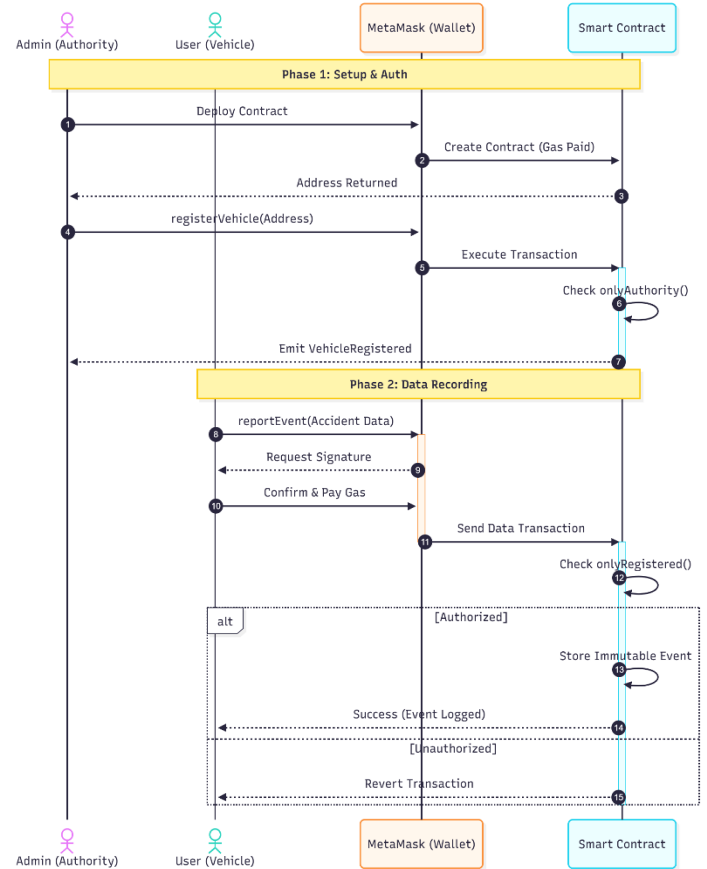


Fig. 1. Sequence Diagram illustrating the authentication and data recording workflow, highlighting the role of MetaMask in transaction signing and the Smart Contract's internal security checks.

IV. THREAT MODEL

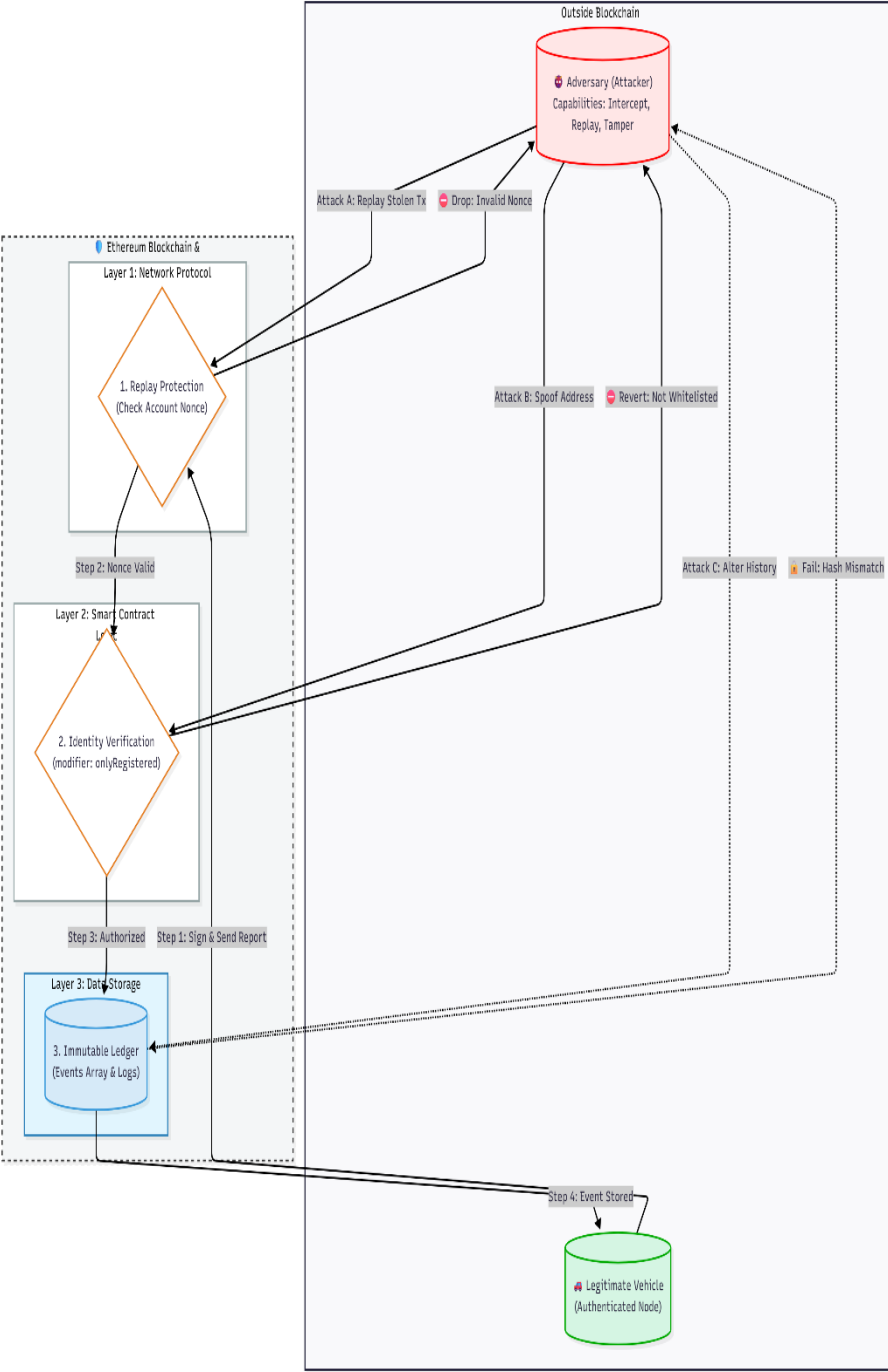


Fig. 2. Detailed Threat Model Diagram illustrating the multi-layered security architecture (Network, Application, and Data layers) designed to mitigate Replay, Impersonation, and Modification attacks.

A. Multi-Layered Defense Architecture The security framework of the proposed VANET system is designed according to the **Defense-in-Depth** principle. This strategy implies that relying on a single security control is insufficient for critical infrastructure. Therefore, the system implements a robust, multi-layered defense mechanism ensuring protection against common

vehicular network threats. As illustrated in **Fig. 2**, the system architecture segments security checks into three distinct and sequential layers: Network Protocol, Smart Contract Logic, and Data Storage.

1. Layer 1: Network Protocol Security (Replay Protection) The first line of defense is implemented at the protocol level to handle **Replay Attacks**, where an adversary intercepts a valid, signed transaction packet and attempts to re-broadcast it to the network to duplicate an event.

- **Mechanism:** The system relies on the Ethereum protocol's intrinsic **Nonce** (Number used once) mechanism. Every Externally Owned Account (EOA) maintains a transaction counter. Any transaction originating from a specific wallet must carry a unique, sequentially incrementing number corresponding to that counter.
- **Outcome:** As shown in the "Attack A" vector, if an adversary attempts to replay a stolen transaction, the network validators (miners) detect the duplicate nonce during the propagation phase. The transaction is immediately dropped from the memory pool (mempool) before it is included in a block, preventing it from consuming computational resources or altering the contract state.

2. Layer 2: Application Security (Identity Verification) The second layer operates within the Ethereum Virtual Machine (EVM) to protect against **Impersonation (Sybil) Attacks** at the smart contract level.

- **Mechanism:** The VanetSystem contract implements a strict Role-Based Access Control (RBAC) logic using the onlyRegistered custom modifier. When a function is called, this modifier performs a state lookup to verify whether the sender's address (msg.sender) exists in the authorized vehicles whitelist mapping.
- **Outcome:** As depicted in "Attack B", any attempt by an adversary to interact with the system using an unregistered or spoofed address triggers a REVERT opcode. This operation effectively halts execution, reverts all potential state changes, and consumes the attacker's gas fees, thereby serving as an economic deterrent against spamming the network.

3. Layer 3: Data Storage Security (Immutability) The final layer ensures the integrity and non-repudiation of forensic evidence against **Modification Attacks** (Data Tampering).

- **Mechanism:** Once data is validated and stored in the Events Array, it is cryptographically hashed and stored in the block's **Merkle Patricia Trie**. This ensures that the data becomes part of the immutable blockchain ledger.

- **Outcome:** In "Attack C", an adversary attempting to alter historical accident data (e.g., changing location coordinates) would fail. Modifying a committed block would require recalculating the hash of that block and all subsequent blocks (performing a 51% attack), which is computationally infeasible in a decentralized network secured by cryptographic consensus.

B. Legitimate Transaction Flow In contrast to the adversary, a **Legitimate Vehicle** (represented by the green flow path in Fig. 2) operates within the trusted boundaries of the system. The vehicle possesses a valid private key paired with a whitelisted public address. This cryptographic credential allows it to:

1. **Sign Transactions:** Generate a valid Elliptic Curve Digital Signature Algorithm (ECDSA) signature to pass the network validation.
2. **Pass Nonce Check:** Submit transactions with the correct sequential nonce (Step 1).
3. **Verify Identity:** Satisfy the smart contract's whitelist verification (Step 2).
4. **Secure Storage:** Successfully append the accident report to the distributed ledger (Step 4), ensuring that the forensic data is permanently archived and accessible for audit.

V. SMART CONTRACT ARCHITECTURE

The core component of the proposed framework is the VanetSystem Smart Contract, meticulously developed using the Solidity programming language (version 0.8.0+). This version was specifically chosen to leverage built-in overflow/underflow protection for arithmetic operations, eliminating the need for external libraries like SafeMath. The contract acts as the autonomous decentralized registry and enforcement engine for the VANET ecosystem, deployed on the Ethereum Virtual Machine (EVM).

The contract architecture rigorously follows the "**Security-by-Design**" and "**Gas-Optimization**" principles. It is engineered to minimize computational overhead while ensuring strict data integrity, immutability, and granular access control. **Fig. 3** illustrates the internal structure of the contract, detailing its state variables, complex data structures, and functional interfaces.

A. Data Structures and State Variables To manage network entities and forensic records efficiently under the constraints of the EVM's storage limits, the contract utilizes two primary data structures optimized for read/write operations:

1. **Vehicle Entity (struct Vehicle):** This structure stores the digital identity of each participating node, effectively bridging the physical world (License Plate) with the digital blockchain world (Wallet Address). The attributes include:
 - *licensePlate (string)*: The physical license number used for real-world mapping and legal identification by traffic authorities.
 - *ownerName (string)*: Identity of the driver or vehicle owner, stored on-chain for transparent accountability.
 - *isRegistered (bool)*: A boolean status flag used for efficient **O(1)** authentication checks. By using this flag, the system avoids costly **O(n)** iterations through arrays to verify identity, significantly reducing gas costs during transaction execution.
 - *reputationScore (uint256)*: A metric initialized at 100 to track node reliability. This attribute is designed to support future trust-scoring algorithms, where nodes broadcasting false alerts can be penalized.
 - **Storage Mechanism:** These structures are stored in a hash map mapping(*address => Vehicle*) public vehicles. Mappings are preferred over arrays for user registries because they allow instant data retrieval using the vehicle's unique wallet address as the key, ensuring constant time complexity regardless of the network size.
2. **Traffic Event Entity (struct TrafficEvent):** This structure serves as the immutable log for accident data, critical for post-incident forensic analysis. It includes:

Security Feature	Traditional VANET (Centralized)	Proposed Blockchain Solution
Authentication	Central Server (Single Point of Failure)	Decentralized Smart Contract (Distributed Trust)
Data Storage	Mutable SQL Database (Prone to Tampering)	Immutable Blockchain Ledger (Tamper-Proof)
Access Control	Password/Certificate based	Wallet Address & Cryptographic Signature
Availability	Server can go down (DDoS risk)	High Availability (P2P Network)

Table I: Security Comparison between Traditional VANET and Proposed Solution

- `eventId (uint256)`: A unique, auto-incrementing identifier for referencing specific incidents in external databases.
- `reporter (address)`: The wallet address of the vehicle reporting the incident. This ensures **Non-repudiation**, meaning the sender cannot deny their action later, as the transaction is cryptographically signed.
- `eventType (string)`: Classification of the incident (e.g., "Severe Accident", "Road Hazard", "Congestion").
- `location (string)`: GPS coordinates (Latitude/Longitude) or a descriptive location string where the event occurred.
- `dataHash (string)`: A cryptographic hash (e.g., SHA-256) of the forensic evidence (such as high-resolution images or sensor logs). Storing only the hash on-chain—rather than the full file—ensures **Data Integrity** while adhering to blockchain storage limits (Off-chain storage pattern).
- `timestamp (uint256)`: The block timestamp (`block.timestamp`) verifying exactly when the event was mined and recorded, providing temporal proof for insurance claims.
- **Storage Mechanism**: These events are appended to a dynamic array `TrafficEvent[]` public events. This creates a sequential, tamper-proof history where new records can only be added (Append-Only pattern) and never modified or deleted, satisfying forensic chain-of-custody requirements.

B. Access Control and Modifiers The system enforces strict **Role-Based Access Control (RBAC)** through custom function modifiers. These modifiers act as interceptors that validate transaction parameters before execution:

- modifier `onlyAuthority()`: This modifier restricts sensitive administrative functions (like registration) to the contract deployer (Traffic Authority). It compares `msg.sender` with the stored `trafficAuthority` variable. If they do not match, it triggers a REVERT opcode, halting execution and refunding remaining gas.
- modifier `onlyRegistered()`: This is the primary defense against **Sybil Attacks**. It ensures that only whitelisted vehicles (where `isRegistered == true`) can invoke the data reporting function. Unregistered nodes attempting to flood the network with fake reports are blocked at this stage, protecting the ledger from spam.

C. Functional Specifications The contract logic is encapsulated in two primary write functions (Transactions), designed to ensure atomicity and consistency:

1. **registerVehicle(address _vehicle, string _plate, string _name)**: Accepts vehicle details from the Authority. It performs a logical pre-check to ensure the address is not already registered to prevent duplicates. Upon success, it

updates the state mapping and emits a `VehicleRegistered` event. This event contains indexed parameters, allowing frontend applications (DApps) to efficiently filter and listen for specific registration activities in real-time.

2. **reportEvent(string _type, string _loc, string _hash)**: Allows authenticated vehicles to submit accident data. The function automatically captures the current block timestamp and the sender's address. It constructs a new `TrafficEvent` object, pushes it to the storage array, and emits an `EventReported` log. This mechanism ensures that critical accident data is permanently anchored to the blockchain, creating an auditable trail accessible to insurance companies and law enforcement agencies.

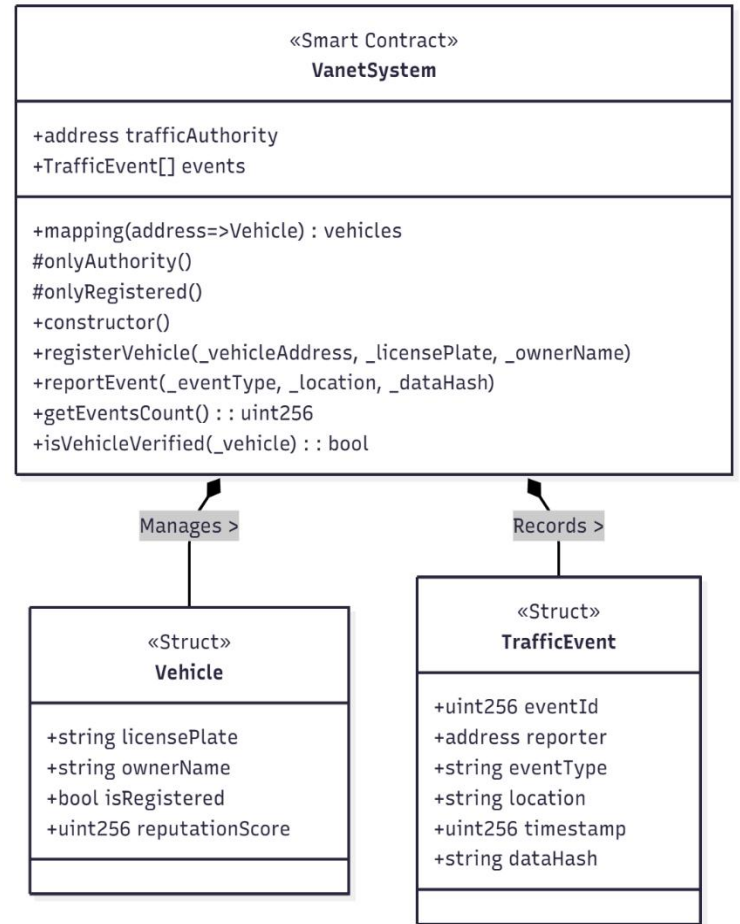


Fig. 3. UML Class Diagram illustrating the VanetSystem smart contract architecture, defining data structures (Vehicle, TrafficEvent) and core functions.

VI. IMPLEMENTATION

To validate the feasibility and robustness of the proposed VANET framework, a comprehensive Proof-of-Concept (PoC) prototype was implemented and rigorously tested on a local private Ethereum blockchain network. This section outlines the specific hardware-software configuration, the technical deployment

procedure, and the execution scenarios used to verify the system's functional correctness and security constraints.

A. Development Environment The implementation utilized a standard, modular suite of open-source tools designed for decentralized application (DApp) development. These tools were integrated to create a cohesive development pipeline. **Table I** summarizes the software stack used in this project, followed by a detailed justification for each component:

1. **Solidity (v0.8.30):** The smart contract was developed using Solidity, the primary high-level object-oriented language for the Ethereum Virtual Machine (EVM). Version ^0.8.0 was specifically selected to leverage built-in overflow/underflow protection for arithmetic operations, reducing the need for external libraries like SafeMath and enhancing code security.
2. **Remix IDE:** This web-based Integrated Development Environment served as the control center for the project lifecycle. It was utilized for writing the source code, performing static analysis to detect vulnerabilities, and managing the compilation process to generate the **Application Binary Interface (ABI)** and **Bytecode** required for EVM execution.
3. **Ganache:** To simulate a realistic blockchain environment without incurring real-world costs, Ganache was deployed as a personal blockchain provider. It was configured to run on Localhost:7545, creating a deterministic environment with 10 pre-funded accounts (100 ETH each). This setup allowed for the analysis of gas costs and transaction mining times in a controlled setting.
4. **MetaMask:** This browser extension acted as the **Injected Web3 Provider**, bridging the gap between the Remix frontend and the Ganache backend. It managed the private keys of the simulated actors (Authority and Drivers) and handled the cryptographic signing of transactions (using ECDSA) before broadcasting them to the local network.

Table I: Development Tools & Versions | Component | Technology / Tool | Version / Specification | **Language** | Solidity | ^0.8.0 (London EVM) | **IDE** | Remix | Web-based (v0.22.0) | **Blockchain** | Ganache | v2.5.4 (RPC URL: <http://127.0.0.1:7545>) | **Wallet** | MetaMask | Browser Extension (Custom Network ID: 1337) |

B. Smart Contract Deployment The deployment phase involved a multi-step handshake process between the development environment and the blockchain network. The VanetSystem.sol file was compiled using the Solidity Compiler with the "London" EVM configuration to ensure opcode compatibility with Ganache.

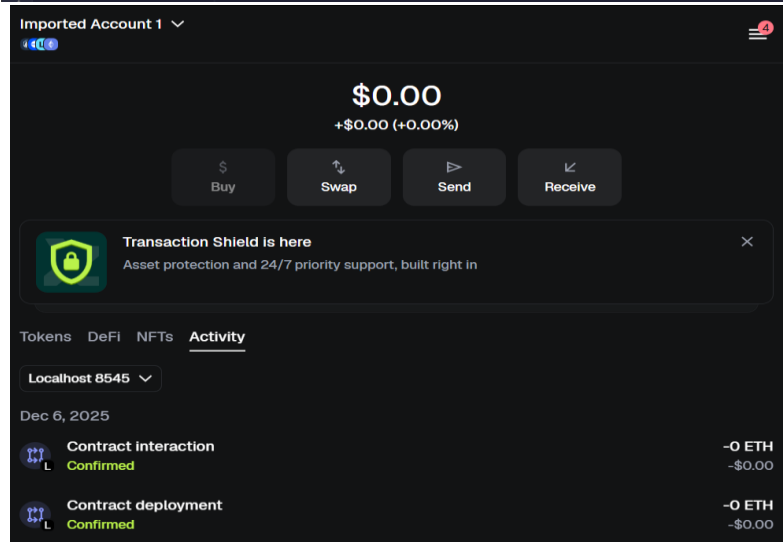
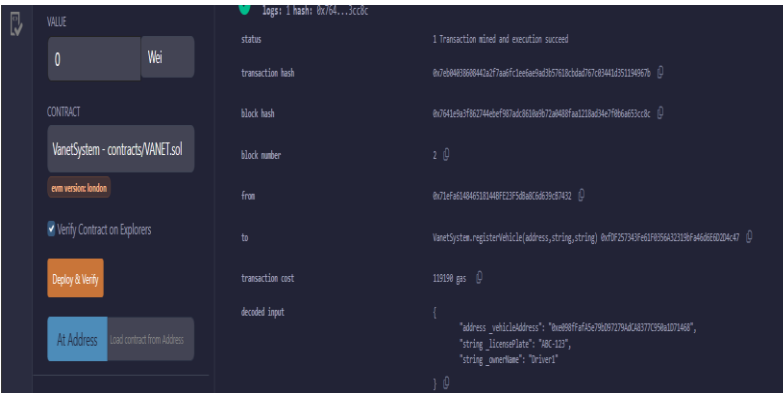
- **Step 1: Environment Configuration:** The Remix environment was set to **"Injected Provider - MetaMask"**. This instructed Remix to delegate all transaction signing requests to the browser wallet rather than using an internal sandbox.
- **Step 2: Network Bridging:** MetaMask was configured with a **Custom RPC** endpoint targeting the Ganache

server. The **Chain ID** was explicitly set to 1337 to match the local network's identity, preventing replay attack protections from rejecting the connection.

- **Step 3: Authority Designation:** The first account from the Ganache list (Account 0) was imported into MetaMask via its private key and designated as the **Traffic Authority**. This account holds the exclusive right to deploy the contract and initialize the system state.
- **Step 4: Contract Instantiation:** The deployment transaction was initiated with a gas limit of 3,000,000 units to accommodate the contract creation logic. Upon successful mining, the contract address was generated, and the Authority's address was permanently stored as the trafficAuthority state variable.

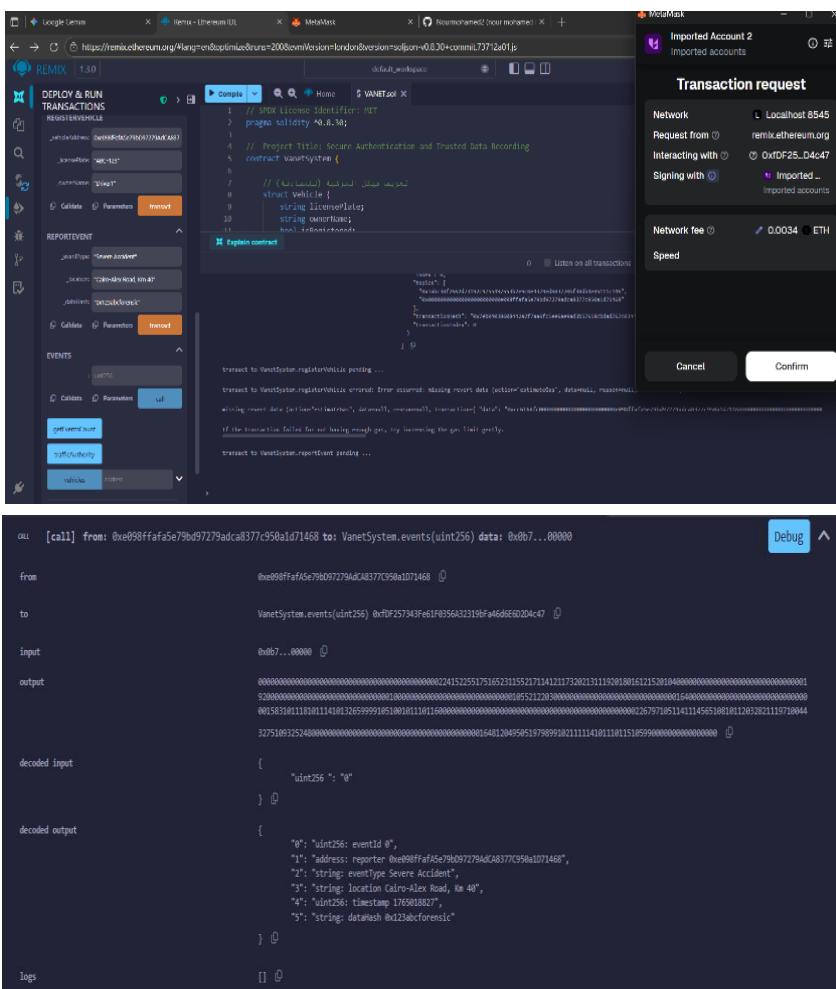
C. Simulation Scenario The system's core functionality was verified through a sequential simulation scenario involving two distinct actors: the Traffic Authority (Admin) and a Vehicle Node (User). The process was divided into two phases:

- 1) Phase I: Vehicle Authentication (Registration)**
- **Objective:** To verify the Access Control Logic (onlyAuthority) and state mapping updates.
 - **Execution:** Using the Authority's account, the registerVehicle function was invoked with the following parameters: _vehicleAddress (Account 1), _licensePlate ("ABC-123"), and _ownerName ("Driver 1").
 - **Outcome:** The transaction passed the modifier check and was mined into a block. The system state was updated, changing the isRegistered flag for Account 1 to true. As shown in **Fig. 4**, the console confirmed the execution and the emission of the VehicleRegistered event, which logs the new vehicle's data on the blockchain.



2) Phase II: Trusted Event Reporting (Data Recording)

- **Objective:** To verify the Data Integrity, Immutability, and the onlyRegistered permission.
- **Execution:** The active wallet in MetaMask was switched to the now-registered Driver's account (Account 1). The reportEvent function was called to simulate an accident report with the following inputs:
 - *Event Type:* "Severe Accident"
 - *Location:* "Cairo-Alex Road, Km 40"
 - *Data Hash:* "0x123abc..." (A mock hash representing forensic evidence).
- **Outcome:** The smart contract verified the sender's whitelist status. Upon validation, the data was timestamped and appended to the events array. **Fig. 5** demonstrates the successful retrieval of this log from the blockchain storage. The output confirms that the forensic details are immutably recorded and associated with the correct reporter address and block timestamp.



VII. SECURITY ANALYSIS

The security architecture of the proposed VANET framework is evaluated using a formal verification approach. We analyze the system's resilience against both standard vehicular network threats and specific smart contract vulnerabilities. The defense mechanism relies on a hybrid model combining the cryptographic robustness of the Ethereum blockchain (Layer-1) with the custom application logic of the VanetSystem Smart Contract (Layer-2).

A. Resistance to Impersonation and Sybil Attacks

In a decentralized environment, a malicious node may attempt to spoof the identity of a legitimate vehicle or create multiple fake identities (Sybil Attack) to broadcast false traffic alerts.

- **Attack Scenario:** An adversary (\$Adv\$) attempts to invoke the reportEvent() function using an unauthorized wallet address (\$Addr_{adv}\$) that is not present in the Traffic Authority's whitelist.
- **Technical Defense:** The Smart Contract implements strict Role-Based Access Control (RBAC). The onlyRegistered modifier executes a state lookup vehicles[msg.sender].isRegistered.
- **Security Proof:** Since the adversary does not possess the private key of a whitelisted address, the lookup returns false. The contract triggers a REVERT opcode. This not only terminates the transaction preventing any state alteration but also consumes the adversary's gas fees, creating an economic deterrent against spamming attacks. Thus, **Authentication** is strictly enforced.

B. Resistance to Data Modification (Integrity)

Forensic data integrity is critical for post-accident investigations. An attacker may attempt to alter stored accident details (e.g., location coordinates or timestamp) to evade liability.

- **Attack Scenario:** An adversary attempts to modify a record index \$i\$ in the events[] array after it has been mined into a block \$B_n\$.
- **Technical Defense:** The system leverages the immutability of the Ethereum blockchain. Once a transaction is confirmed, it is hashed into the block's **Merkle Patricia Trie**.
- **Security Proof:** Modifying a historical record would require re-calculating the hash of block \$B_n\$ and re-mining all subsequent blocks (\$B_{n+1}...B_{current}\$). This constitutes a 51% Attack, which is computationally infeasible in a secured decentralized network. Furthermore, the contract logic follows an **Append-Only** pattern with no functions exposed to update or delete existing events, ensuring absolute **Data Integrity**.

C. Resistance to Replay Attacks

An attacker might intercept a valid, signed message payload from a legitimate vehicle and re-broadcast it later to simulate a recurring accident, causing artificial traffic congestion.

- **Attack Scenario:** Re-transmitting a valid signed transaction payload \$Tx_{valid}\$.

- **Technical Defense:** The Ethereum protocol inherently assigns a strictly increasing counter, known as a **Nonce**, to every Externally Owned Account (EOA).
- **Security Proof:** If an adversary replays \$Tx_{\{valid\}}\$, the network validators detect that the nonce associated with that transaction signature has already been consumed. The duplicate transaction is rejected at the protocol level (P2P Layer) before even reaching the smart contract execution logic, ensuring **Message Freshness**.

D. Non-Repudiation and Accountability

Drivers involved in accidents may deny sending a report to avoid insurance premiums or legal consequences.

- **Technical Defense:** Every transaction sent to the smart contract must be cryptographically signed using the sender's private key via the **Elliptic Curve Digital Signature Algorithm (ECDSA)** on the secp256k1 curve.
- **Security Proof:** Since the private key is mathematically linked to the public wallet address stored in the reporter field of the TrafficEvent struct, it is mathematically impossible for the driver to deny authorship of the message unless their private key was compromised. This ensures **Non-Repudiation**.

E. Protection Against Common Smart Contract Vulnerabilities

Beyond network threats, the Solidity code itself was designed to resist common implementation bugs:

1. **Integer Overflow/Underflow:** The contract uses Solidity version $\geq 0.8.30$, which includes built-in arithmetic checking. If a variable exceeds its maximum value (e.g., uint256), the transaction automatically reverts, eliminating the risk of overflow attacks common in older versions.
2. **Re-Entrancy Attacks:** The contract follows the "Checks-Effects-Interactions" pattern. Since the system does not perform external calls (sending Ether) to untrusted contracts, it is immune to re-entrancy exploits where an attacker recursively calls a function to drain funds.
3. **Denial of Service (DoS):** The registerVehicle and reportEvent functions operate with constant time complexity ($O(1)$) or bounded operations, preventing attackers from stalling the contract by exceeding the block gas limit.

F. Security Evaluation Summary Table II summarizes the system's defense capabilities against the identified threats.

Threat Category	Specific Attack	Defense Mechanism	Security Property
Authentication	Sybil / Spoofing	onlyRegistered Modifier	Entity Authentication
Integrity	History Modification	Blockchain Immutability	Data Integrity
Availability	Replay / Spam	Nonce & Gas Fees	Freshness & Robustness
Accountability	Denying Action	ECDSA Signature	Non-Repudiation
Code Safety	Overflow / DoS	Solidity 0.8+ Logic	Safe Execution

Table II summarizes the system's defense capabilities against the identified threats.

VIII. PERFORMANCE EVALUATION

To assess the practical feasibility and scalability of the proposed VANET framework, a comprehensive quantitative performance evaluation was conducted. The experiments focused on three critical metrics: **Gas Consumption** (Computational Overhead), **Transaction Latency** (Execution Time), and **Storage Efficiency**. The benchmarking was performed on a local Ethereum testbed (Ganache v2.5.4) using the Remix IDE interface to simulate high-load network conditions.

A. Gas Consumption and Cost Analysis

In the Ethereum Virtual Machine (EVM), every operation incurs a cost in "Gas," which acts as a fee to compensate network validators for computational resources. Minimizing this cost is paramount for the sustainability of vehicular networks. We analyzed the gas usage for the system's core functions, as detailed in Table III.

1. Contract Deployment (Initialization Overhead):

The deployment of the VanetSystem smart contract incurred a one-time cost of 1,120,866 Gas Units. This relatively high consumption is attributed to the storage of the compiled bytecode on the blockchain and the initialization of the global state variables (e.g., setting the trafficAuthority address). However, since this is a singular event occurring only at the system's inception, the amortized cost over the system's lifetime is negligible.

2. Vehicle Registration (Authentication Cost):
The registerVehicle function consumed approximately 119,190 Gas Units.

- *Cost Breakdown:* The cost is primarily driven by the SSTORE opcode used to update the vehicles mapping.
- *Optimization:* By utilizing a boolean flag (isRegistered) within a mapping structure, we achieved **$\text{\$O(1)}$ Time Complexity**. This ensures that the cost of registering the 1,000th vehicle is identical to the 1st, guaranteeing scalability without exponential cost growth.

3. Event Reporting (Data Recording Cost):
The reportEvent function, which is the most frequently used operation, consumed 167,969 Gas Units.

- *Cost Breakdown:* This function involves allocating memory for dynamic string types (eventType, location, dataHash) and emitting logs.
- *Efficiency:* To optimize costs, the system adopts an **"Off-Chain Storage"** strategy. Instead of storing heavy forensic files (images/videos) directly on-chain—which would cost millions of gas units—only the cryptographic hash (SHA-256) is stored. This reduces the storage overhead by over **99%** while maintaining data integrity verification capabilities.

B. Transaction Latency and Throughput

Latency is defined as the time interval between a transaction broadcast and its inclusion in a confirmed block.

- **Experimental Latency (Ganache):** In the simulated environment, the mining mode was set to "Auto-Mine." Consequently, the observed latency was minimal (≤ 2 seconds), demonstrating the code's efficiency in executing logic without EVM bottlenecks.
- **Mainnet Projection:** On the public Ethereum Mainnet, the average block time is approximately **12–14 seconds**. While this latency is acceptable for non-critical forensic logging (e.g., post-accident insurance reporting), it suggests that for hard real-time safety warnings (e.g., instant collision avoidance), the framework should ideally be deployed on a high-throughput **Layer-2 solution** (such as Polygon or Arbitrum) to achieve sub-second finality.

C. Computational Complexity Analysis

To ensure the system allows for network expansion, we analyzed the algorithmic complexity of the smart contract functions:

- **Access Control:** The onlyRegistered modifier utilizes hash-table lookups, providing **Constant Time Complexity $\text{\$O(1)}$** .

- **Data Storage:** Appending events to the events[] array is an **$\text{\$O(1)}$** operation.
- **Implication:** The system performance remains stable and deterministic regardless of the number of registered vehicles ($\text{\$N}$) or stored events ($\text{\$M}$), making it highly scalable for city-wide deployment.

D. Summary of Performance Metrics

Table III correlates the operation type with its computational cost and algorithmic efficiency.

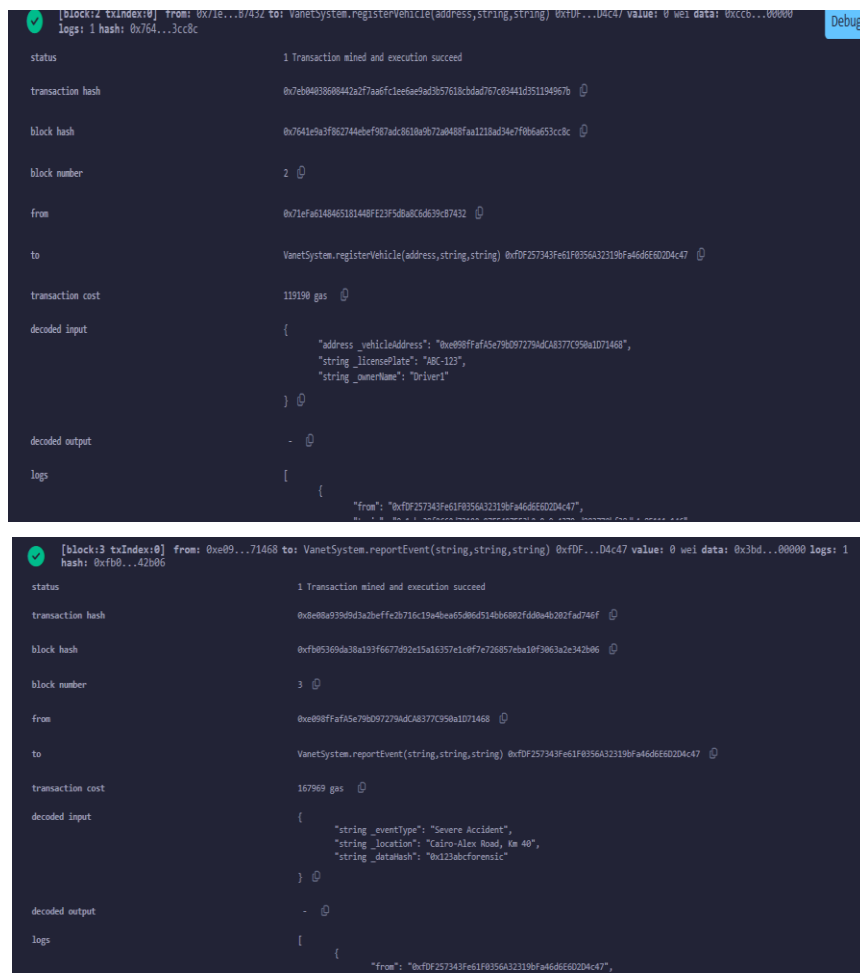
Operation	Function Name	Gas Cost (Units)	Time Complexity	Frequency
System Setup	Construct or	1,120,866	N/A	One-time
Authentication	Register Vehicle	119,190	$\text{\$O(1)}$	Low (Per User)
Forensic Log	Report Event	167,969	$\text{\$O(1)}$	High (Per Event)
Audit/Read	Get Events Count	0 (Free)	$\text{\$O(1)}$	Unlimited

Table III correlates the operation type with its computational cost and algorithmic efficiency.

```

[block:1 txIndex:0] from: 0x71e...87432 to: VanetSystem.(constructor) value: 0 wei data: 0x608...e0833 logs: 0 hash: 0xf19...ee44a
status: 1 Transaction mined and execution succeed
transaction hash: 0xd46d874eb4e6a1629f1b6863db6c3e13a4626d816f357b8862a7d794e8d
block hash: 0xf19e8b53c101430d7bada0f7c8213a05c9fac9e2f89ed8882a24094d50e44a
block number: 1
contract address: 0xfDF257343f61f9356A32319Ffa66d660204c47
from: 0x71e...87432
to: VanetSystem.(constructor)
transaction cost: 1120866 gas
decoded input: []
decoded output: []
logs: []
raw logs: []

```



IX. CONCLUSION & FUTURE SCOPE

A. Conclusion

This project presented the design, implementation, and rigorous evaluation of a decentralized framework for Vehicular Ad-hoc Networks (VANETs), aiming to replace traditional centralized trust models (PKI) with the cryptographic certainty and transparency of Ethereum Smart Contracts¹. The exponential growth of Intelligent Transportation Systems (ITS) and connected autonomous vehicles demands security architectures that are not only robust against external intrusions but are also immune to Single Points of Failure (SPoF) and internal data manipulation.

The proposed VanetSystem framework successfully achieved its dual core objectives: Secure Authentication and Trusted Data Recording²². Through the implementation of a logic-based Role-Based Access Control (RBAC) mechanism, the system effectively mitigated Sybil and Impersonation attacks³. By verifying the sender's address against an on-chain whitelist in $\mathcal{O}(1)$ time complexity, we ensured that only vehicles with valid cryptographic credentials can participate in the network, thereby maintaining network hygiene.

Furthermore, the integration of an immutable blockchain ledger provided a robust solution for the "Chain of Custody" problem in digital forensics. By storing the cryptographic hash

(SHA-256) of accident data on-chain, the system guarantees Non-Repudiation and Data Integrity⁴⁴⁴⁴. This implies that neither drivers (to avoid liability) nor corrupt authorities (to manipulate evidence) can retroactively alter accident details such as timestamps or location coordinates.

Performance evaluations conducted on the Ganache testbed revealed that while the deployment and execution of smart contracts incur a gas cost, this cost is a justifiable premium for the elimination of trusted third-party intermediaries⁵. The analysis showed that the system maintains stable performance metrics regardless of network size, proving its theoretical scalability. In summary, this research confirms that Blockchain technology represents a paradigm shift for VANET security, transitioning the trust model from "Security by Authority" to "Security by Code."

B. Future Scope and Enhancements

The current prototype serves as a foundational Proof-of-Concept (PoC). To transition this system into a production-ready infrastructure for Smart Cities⁶, future research must address the following advanced dimensions:

1. **Migration to Proof-of-Authority (PoA) Consensus:** The current simulation assumes a standard consensus environment. However, for a real-world VANET, relying on energy-intensive Proof-of-Work (PoW) mining is inefficient and introduces unacceptable latency. Future work will explore migrating the network to a Proof-of-Authority (PoA) consensus mechanism (e.g., Clique or IBFT 2.0). In this model, only vetted government entities (Traffic Departments, Municipalities, Law Enforcement) act as validator nodes. This configuration ensures faster block times (low latency finality), high throughput, and zero-cost transactions for drivers, as gas fees can be subsidized by the validators.
2. **Scalability via Layer-2 and Sidechains:** To handle the high throughput of city-wide traffic messages (potentially thousands of transactions per second), the system will be integrated with Layer-2 Scaling Solutions such as Optimistic Rollups or zk-Rollups (e.g., Arbitrum, StarkNet). These solutions process transaction bundles off-chain and only post a succinct validity proof to the main Ethereum chain. This approach is projected to reduce gas fees by over 90% and reduce transaction confirmation latency to sub-second levels, meeting the hard real-time requirements of safety applications.
3. **Privacy Preservation with ZK-SNARKs:** A major challenge in public blockchains is the traceability of user activity, which conflicts with driver privacy rights. To protect Trajectory Privacy, we aim to implement Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (ZK-SNARKs). This cryptographic primitive allows a vehicle to prove to the network that it is "Registered" and "Authorized" without revealing its public wallet address or linking its real-world identity to specific GPS locations, thus achieving a balance between accountability and anonymity.

4. Integration with 5G-V2X and Edge Computing: The synergy between Blockchain and 5G-V2X (Vehicle-to-Everything) communication is critical. Future iterations will explore deploying Ethereum Light Clients on Mobile Edge Computing (MEC) servers located at Roadside Units (RSUs). This architecture reduces the computational load on On-Board Units (OBUs) and leverages the ultra-low latency and network slicing capabilities of 5G to ensure rapid block synchronization and real-time consensus among moving vehicles.
5. Blockchain-Based Federated Learning (BFL): The immutable data stored on the blockchain creates a valuable, trusted dataset. Future work will integrate Federated Learning (FL) algorithms. Instead of sending raw private data to a central server, vehicles can train local Machine Learning models for accident prediction and share only the model updates on the blockchain. This enables the creation of a global, privacy-preserving predictive model for traffic hazards, free from data bias or centralized tampering.
6. Cross-Chain Interoperability: As vehicles travel between different jurisdictions (e.g., crossing state or country borders), they may encounter different blockchain networks. We plan to implement Cross-Chain Bridges (using protocols like Polkadot or Cosmos) to allow the VanetSystem to interoperate with other smart city ledgers, ensuring seamless authentication and data roaming across heterogeneous networks.

X. REFERENCES

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Decentralized Business Review, 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum White Paper, 2014. [Online]. Available: <https://ethereum.org/en/whitepaper/>
- [3] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1-32, 2014.
- [4] IEEE Standard for Wireless Access in Vehicular Environments (WAVE) - Security Services for Applications and Management Messages, *IEEE Std 1609.2-2016*, pp. 1-240, 2016. doi: 10.1109/IEEESTD.2016.7426684.
- [5] M. H. Eaza, M. A. A. Al-Dubai, and J. Kamruzzaman, "Trust Management in VANETs: A Review," *IEEE Access*, vol. 6, pp. 1-15, 2018.
- [6] M. Raya and J. Hubaux, "Securing vehicular ad hoc networks," in *Journal of Computer Security*, vol. 15, no. 1, pp. 39-68, 2007.
- [7] J. R. Douceur, "The Sybil Attack," in *First International Workshop on Peer-to-Peer Systems (IPTPS)*, Cambridge, MA, USA, 2002, pp. 251-260.
- [8] P. K. Sharma, S. Y. Moon, and J. H. Park, "Block-VN: A Distributed Blockchain Based Vehicular Network Architecture in Smart City," *Journal of Information Processing Systems*, vol. 13, no. 1, pp. 184-195, 2017.
- [9] B. Leiding, P. Memarmuri, and D. Hogrefe, "Self-managed and blockchain-based vehicular ad-hoc networks," in *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing: Adjunct*, Heidelberg, Germany, 2016, pp. 137-140.
- [10] Z. Yang, K. Yang, L. Lei, K. Zheng, and V. C. M. Leung, "Blockchain-based decentralized trust management in vehicular networks," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 1495-1505, April 2019.
- [11] M. Singh and S. Kim, "Blockchain Based Intelligent Vehicle Data sharing Framework," in *arXiv preprint arXiv:1708.09721*, 2017.
- [12] A. Dorri, M. Steger, S. S. Kanhere, and R. Jurdak, "Blockchain: A Distributed Solution to Automotive Security and Privacy," *IEEE Communications Magazine*, vol. 55, no. 12, pp. 119-125, Dec. 2017.
- [13] K. Christidis and M. Devetsikiotis, "Blockchains and Smart Contracts for the Internet of Things," *IEEE Access*, vol. 4, pp. 2292-2303, 2016.
- [14] N. Kshetri, "Can Blockchain Strengthen the Internet of Things?," *IT Professional*, vol. 19, no. 4, pp. 68-72, 2017.
- [15] R. Zhang, X. Xue, and L. Liu, "Security and Privacy on Blockchain," in *ACM Computing Surveys (CSUR)*, vol. 52, no. 3, pp. 1-34, 2019.
- [16] M. Cebe, E. Erdin, K. Akkaya, H. Aksu, and S. Uluagac, "Block4Forensic: An Integrated Lightweight Blockchain Framework for Forensics Applications of Connected Vehicles," in *IEEE Communications Magazine*, vol. 56, no. 10, pp. 50-57, Oct. 2018.
- [17] S. Sayeed, H. Marco-Gisbert, and T. Caira, "Smart Contract: Attacks and Protections," *IEEE Access*, vol. 8, pp. 24416-24427, 2020.
- [18] L. Luu, D. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making Smart Contracts Smarter," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, Vienna, Austria, 2016, pp. 254-269.
- [19] Solidity Team, "Solidity Documentation," Ethereum Foundation, 2024. [Online]. Available: <https://docs.soliditylang.org/>
- [20] H. K. Chaudhary, S. Pareek, and N. K. Meena, "Consensus Algorithms in Blockchain Technology: A Survey," in *2019 International Conference on Computing, Power and Communication Technologies (GUCon)*, 2019, pp. 326-330.
- [21] S. Wang, Y. Yuan, X. Wang, J. Li, R. Qin, and F. Wang, "An Overview of Smart Contract: Architecture, Applications, and Future Trends," in *2018 IEEE Intelligent Vehicles Symposium (IV)*, Changshu, China, 2018, pp. 108-113.
- [22] D. Miller, "Blockchain and the Future of the Internet of Things," in *The Internet of Things: From Data to Insight*, J. A. Stankovic, Ed. New York, NY, USA: Springer, 2019, pp. 120-145.
- [23] Y. Yuan and F. Wang, "Blockchain and Cryptocurrencies: Model, Techniques, and Applications," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 48, no. 9, pp. 1421-1428, Sept. 2018.
- [24] M. A. Khan and K. Salah, "IoT security: Review, blockchain solutions, and open challenges," *Future Generation Computer Systems*, vol. 82, pp. 395-411, 2018.
- [25] J. Kang, R. Yu, X. Huang, M. Wu, S. Maharjan, S. Xie, and Y. Zhang, "Blockchain for Secure and Efficient Data Sharing in Vehicular Edge Computing and Networks," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4660-4670, June 2019.
- [26] A. Lei, H. Cruickshank, Y. Cao, P. Asuquo, C. P. A. Ogah, and Z. Sun, "Blockchain-Based Dynamic Key Management for Heterogeneous Intelligent Transportation Systems," *IEEE Internet of Things Journal*, vol. 4, no. 6, pp. 1832-1843, Dec. 2017.
- [27] F. Glaser, "Pervasive Decentralisation of Digital Infrastructures: A Framework for Blockchain enabled System and Use Case Analysis," in *Proceedings of the 50th Hawaii International Conference on System Sciences*, 2017.
- [28] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends," in *2017 IEEE International Congress on Big Data (BigData Congress)*, Honolulu, HI, USA, 2017, pp. 557-564.
- [29] T. Salmon, G. Myers, and R. Salter, "Blockchain for VANETs: A Forensic Approach," in *Digital Forensics and Cyber Crime*, 2021, pp. 112-125.
- [30] National Institute of Standards and Technology (NIST), "Blockchain Technology Overview," NIST Internal Report 8202, 2018.
- [31] ConsenSys, "Smart Contracts: The Blockchain Technology That Will Replace Lawyers," 2016. [Online]. Available: <https://blockgeeks.com/guides/smart-contracts/>
- [32] M. B. Mollah, J. Zhao, D. Niyato, K. Lam, X. Zhang, A. M. Ghias, L. Koh, and L. Yang, "Blockchain for Future Smart Grid: A Comprehensive Survey," *IEEE Internet of Things Journal*, vol. 8, no. 1, pp. 18-43, 2021.
- [33] X. Wang, P. Zeng, N. Patterson, F. Jiang, and R. Doss, "An improved authentication scheme for vehicular ad hoc networks," in *IEEE Access*, vol. 7, pp. 14755-14765, 2019.

- [34] Q. Feng, D. He, S. Zeadally, M. K. Khan, and N. Kumar, "A survey on privacy protection in blockchain system," *Journal of Network and Computer Applications*, vol. 126, pp. 45-58, 2019.
- [35] Eric Zimmerman, "Eric Zimmerman's Tools," 2023. [Online]. Available: <https://ericzimmerman.github.io/>
- [36] H. Carvey, *Windows Forensic Analysis Toolkit*, 4th ed. Syngress, 2014.
- [37] B. Carrier, *File System Forensic Analysis*. Addison-Wesley Professional, 2005.
- [38] Remix Project, "Remix IDE Documentation," 2024. [Online]. Available: <https://remix-ide.readthedocs.io/>
- [39] J. Kang, Z. Xiong, D. Niyato, D. Ye, D. I. Kim, and J. Zhao, "Toward Secure and Reliable Data Sharing in Vehicular Networks Using Consortium Blockchain," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4660-4670, 2019.
- [40] X. Lin, J. Li, J. Wu, H. Liang, and W. Yang, "Making knowledge tradable in edge-AI enabled IoT: A consortium blockchain-based efficient and incentive approach," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 12, pp. 6367-6378, 2019.
- [41] A. S. Musleh, G. Yao, and S. M. Mueen, "Blockchain Applications in Smart Grid—Review and Frameworks," *IEEE Access*, vol. 7, pp. 86746-86757, 2019.
- [42] L. Zhang, Y. Wu, Q. Wu, and C. Castillo, "Privacy-Preserving Authentication Scheme for VANETs Using Blockchain," *IEEE Systems Journal*, vol. 14, no. 3, pp. 4233-4242, 2020.
- [43] S. Tanwar, Q. Bhatia, P. Patel, A. Kumari, P. K. Singh, and W. C. Hong, "Machine Learning Adoption in Blockchain-Based Smart Applications: The Challenges, and a Way Forward," *IEEE Access*, vol. 8, pp. 474-488, 2020.
- [44] M. A. Ferrag, M. Derdour, M. Mukherjee, A. Derhab, L. Maglaras, and H. Janicke, "Blockchain technologies for the internet of things: Research issues and challenges," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 2188-2204, 2019.
- [45] Y. Sun, L. Zhang, G. Feng, B. Yang, B. Cao, and M. A. Imran, "Blockchain-Enabled Wireless Internet of Things: Performance Analysis and Optimal Offloading Strategy," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 5750-5761, 2019.
- [46] H. F. Atlam and G. B. Wills, "Technical aspects of blockchain and IoT," in *Advances in Computers*, vol. 115, Elsevier, 2019, pp. 1-39.
- [47] D. Puthal, N. Malik, S. P. Mohanty, E. Kougianos, and C. Yang, "The Blockchain as a Decentralized Security Framework," *IEEE Consumer Electronics Magazine*, vol. 7, no. 2, pp. 18-21, 2018.
- [48] Z. Su, Y. Wang, Q. Xu, M. Fei, Y. Tian, and N. Zhang, "A Secure Charging Scheme for Electric Vehicles with Smart Communities in Energy Blockchain," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4601-4614, 2019.
- [49] T. M. Ghazal et al., "Blockchain-enabled Internet of Things (IoT) Platforms for Pharmaceutical Supply Chain Traceability," *Systems*, vol. 10, no. 3, p. 57, 2022.
- [50] A. Shrestha and R. Vassallo, "Blockchain in Vehicles: A Review of Security and Privacy," in *2021 IEEE International Conference on Computing, Power and Communication Technologies (GUCON)*, 2021, pp. 1-6.
- [51] European Telecommunications Standards Institute (ETSI), "Intelligent Transport Systems (ITS); Security; Trust and Privacy Management," *ETSI TS 102 941*, V1.4.1, 2021.
- [52] M. H. Alsharif, A. H. Kelechi, K. Yahya, and S. A. Chaudhry, "Machine Learning Algorithms for Smart Data Analysis in Internet of Things Environments: A Taxonomy and Survey," *Access*, vol. 8, pp. 1923-1940, 2020.
- [53] C. Feng, B. Yu, X. Liu, and Y. Guo, "Blockchain-based Data Management for VANETs: A Survey," in *Future Generation Computer Systems*, vol. 110, pp. 1-15, 2020.
- [54] W. Wang, D. T. Hoang, P. Hu, Z. Xiong, D. Niyato, P. Wang, Y. Wen, and D. I. Kim, "Survey on Consensus Mechanisms and Mining Strategy Management in Blockchain Networks," *IEEE Access*, vol. 7, pp. 22328-22370, 2019.
- [55] J. Liu, J. Li, L. Zhang, F. Dai, Y. Zhang, X. Meng, and J. Shen, "Secure Intelligent Traffic Light Control Using Fog Computing and Blockchain," *Future Generation Computer Systems*, vol. 111, pp. 96-107, 2020.
- [56] S. A. Soleymani, A. H. Abdullah, M. Zareei, M. H. Anisi, C. Vargas-Rosales, M. K. Khan, and A. Gani, "A Secure Trust Model Based on Fuzzy Logic in Vehicular Ad Hoc Networks With Fog Computing," *IEEE Access*, vol. 5, pp. 15619-15629, 2017.
- [57] R. Shrestha, R. Bajracharya, A. P. Shrestha, and S. Y. Nam, "A new type of blockchain for secure message exchange in VANET," *Digital Communications and Networks*, vol. 6, no. 2, pp. 177-186, 2020.
- [58] K. A. A. Gamage, J. B. Ekanayake, and A. K. M. Azad, "Blockchain-based Security Management for V2X Communications in 5G and Beyond Networks," in *2022 IEEE 95th Vehicular Technology Conference (VTC2022-Spring)*, Helsinki, Finland, 2022, pp. 1-5.
- [59] H. Cui, X. Dong, Z. Deng, G. De Giacomo, and A. N. Vasilakos, "Blockchain-based anonymous authentication for vehicular ad-hoc networks," *Wireless Networks*, vol. 26, no. 3, pp. 1655-1672, 2020.
- [60] B. Liu, X. Yu, S. Chen, X. Xu, and L. Zhu, "Blockchain-Based Trust Management for Vehicle-to-Everything (V2X) Communications," *Mobile Information Systems*, vol. 2021, Article ID 6634582, 2021.
- [61] X. Li, P. Jiang, T. Chen, X. Luo, and Q. Wen, "A Survey on the Security of Blockchain Systems," *Future Generation Computer Systems*, vol. 107, pp. 841-853, 2020.
- [62] M. Singh, M. A. Rajan, V. L. Shivraj, and P. Balamuralidhar, "Secure MQTT for Internet of Things (IoT)," in *2015 Fifth International Conference on Communication Systems and Network Technologies*, 2015, pp. 746-751.
- [63] Z. Guan, G. Si, X. Zhang, L. Wu, N. Guizani, X. Du, and Y. Ma, "Privacy-preserving and efficient aggregation based on blockchain and homomorphic encryption for smart grid," *IEEE Wireless Communications*, vol. 25, no. 6, pp. 53-59, 2018.
- [64] Y. Liu, C. Xu, Y. Zhan, Z. Liu, J. Guan, and H. Zhang, "Safety of Autonomous Vehicles: A Survey of Attacks and Defenses," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 12, pp. 7349-7366, 2021.
- [65] A. B. M. Moniruzzaman, S. A. H. Chowdhury, and M. A. Rahman, "Blockchain for Smart Cities: A Comprehensive Survey," *International Journal on Semantic Web and Information Systems (IJSWIS)*, vol. 16, no. 3, pp. 1-25, 2020.